

4주차 연습과제

[4장]

- GBM
- XGBoost
- LightGBM
- 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝
- 분류 실습 - 캐글 신용카드 사기 검출
- 스택킹 앙상블

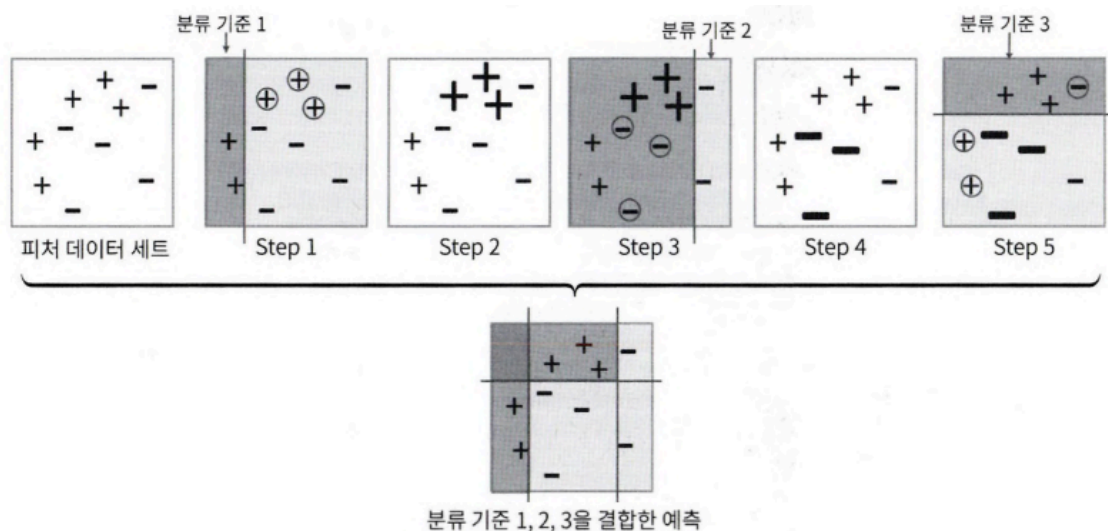
5) GBM

GBM의 개요 및 실습

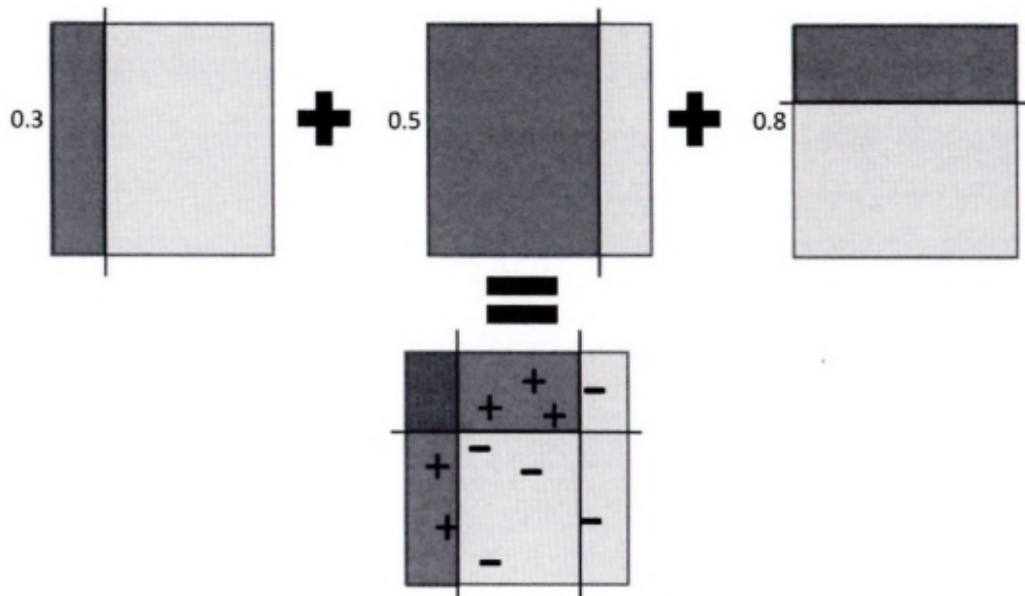
부스팅 알고리즘: 여러 개의 약한 학습기를 순차적으로 학습-예측하며, 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가는 방식

AdaBoost(Adaptive Boosting): 오류 데이터에 가중치를 부여하면서 부스팅을 수행하는 대표적 알고리즘

순서-첫 번째 학습기가 분류 후 오류 데이터 발생 → 오류 데이터에 가중치 부여 → 다음 학습기가 더 잘 분류할 수 있게 크기 조정 → 모든 예측 결정 기준을 결합해 최종 예측 수행



GBM(Gradient Boost Machine): AdaBoost와 유사하나, 가중치 업데이트에 경사하강법(Gradient Descent)을 이용하는 것이 차이점



경사하강법: 반복 수행을 통해 오류식 $h(x) = y - F(x)$ 을 최소화하는 방향으로 가중치 값을 업데이트 하는 방식(실제 값 y , 피쳐 $x \rightarrow$ 피쳐 예측 함수 $F(x)$)

GBM의 특징 및 수행 시간수행 성능: 일반적으로 랜덤 포레스트보다 예측 성능이 뛰어난 경우가 많음.

```
gb_clf = GradientBoostingClassifier(random_state=0)
gb_clf.fit(X_train, y_train)
gb_pred = gb_clf.predict(X_test)
gb_accuracy = accuracy_score(y_test, gb_pred)
print('GBM 정확도 : {0:.4f}'.format(gb_accuracy))
print('GBM 정확도 : {0:.1f}초'.format(time.time() - start_time))
```

→ GBM 정확도 : 0.9389

GBM 수행 시간 : 537.7초

+) 일반적으로 랜덤 포레스트보다는 나은 예측 성능을 보이지만 수행 시간 ↑, 하이퍼 파라미터 튜닝 필요

GBM 하이퍼 파라미터 소개

- 트리 기반 자체 파라미터: `max_depth`, `max_features`
 - `loss`: gradient descent에서 사용할 비용 함수 지정 (default=deviance)
 - `learning_rate`: 학습을 진행할 때마다 적용하는 학습률
 - 0~1 사이 값 (default=0.1)
 - 너무 작은 값: 업데이트 값 ↓ → 최소 오류 값을 찾아 성능↑/수행 시간↑, 모든 weak learner 반복 완료해도 최소 오류 값 찾지 X
 - 너무 큰 값: 수행 시간↓/최소 오류 값 찾지 못하고 지나쳐 성능 ↓
 - `n_estimators` 와 상호 보완적으로 조합!
 - learning_rate ↓, n_estimators ↑ : 성능 한계점까지는 예측 성능이 조금씩 좋아지지만 수행 시간 ↑, 성능이 현격히 좋아지는 수준은 X
 - `n_estimators`: weak learner의 개수 (default=100)
 - 많을수록 순차적으로 오류를 보정해 일정 수준까지는 성능↑/수행시간↑
 - `subsample`: weak learner가 학습에 사용하는 데이터의 샘플링 비율 (default =1, 학습 데이터의 100%를 기반으로 학습)
 - 과적합 염려 시 1 미만으로 설정한다
-

6) XGBoost(eXtra Gradient Boost)

XGBoost 개요

XGBoost: 트리 기반 앙상블 학습에서 유명하며 GBM에 기반하지만, 수행시간 증가 및 과적합 규제 부재 문제를 해결+병렬 CPU 환경에서 병렬 학습 가능, 속도↑

[장점]

1. 뛰어난 예측 성능: 분류/회귀서 뛰어남
2. GBM 대비 빠름: XGBoost(병렬) ↔ GBM(순차적)
3. 과적합 규제 Regularization: 자체 과적합 규제 기능
4. Tree pruning (가지치기): 더 이상 긍정 이득이 없는 분할을 가지치기 + `max_depth` 파라미터 → 분할 수
더 줄임(GBM도 분할 시 부정 손실 발생하면 분할 중지하지만, 그래도 과잉 분할 발생 가능성 존재)
5. 자체 내장된 교차검증: 반복 때마다 train set과 test set 에 대해 교차 검증 수행 → 최적화된 반복 수

행 횟수 가지며 평가 값이 최적화되면 지정 반복 횟수 전 반복 중지 가능(= 조기 중단 가능)

6. 결손값 자체 처리

사이킷런과 호환되는 Wrapper class 제공(이하 사이킷런 래퍼 `XGBoost`): `XGBClassifier`, `XGBRegressor`

파이썬 래퍼 XGBoost 하이퍼 파라미터

1. **일반 파라미터**: 스레드 개수나 silent모드 등의 선택을 위한 파라미터, 디폴트 값을 바꾸는 경우 거의X

- `booster` : gbtree(tree based model) 또는 gblinear(linear model) 선택 (default=gbtree)
- `silent` : 출력 메시지 나타내고 싶지 않을 때 1 (default=0)
- `nthread` : CPU의 실행 스레드 개수 조정 (default=CPU의 전체 스레드 모두 사용)

2. **부스터 파라미터**: 트리 최적화, 부스팅, Regularization 등의 관련 파라미터 지칭하며, 대부분의 파라미터가 여기에 속한다

- `eta` : GBM의 learning rate와 동일하며 0~1 사이 값 지정 (default=0.3, alias: learning_rate)
- `num_boost_rounds` : GBM의 n_estimators 와 동일
- `min_child_weight` : 트리에서 추가적으로 데이터를 나눌지 결정하기 위해 필요한 D들의 weight 총합으로 클수록 분할 자제(default=1))
- `gamma` : 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값, 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드 분리/클수록 과적합 감소 (default=0, alias: min_split_loss)
- `max_depth` : 0 지정 시 깊이 제한 X, 높을수록 조건 특화된 룰 조건 생성 → 과적합 가능, 보통 3~10 값 사용(default=6)
- `sub_sample` : 트리가 커져 과적합되는 것을 방지하기 위해 데이터를 샘플링하는 비율 지정, 일반적으로 0.5~1 값 사용 (0.5 = 전체 50%를 트리 생성에 사용), (dafault=1)
- `colsample` : GBM의 max_features와 유사하며, 트리 생성에 필요한 피처(칼럼)을 임의로 샘플링 (default=1, alias: reg_lambda)
- `lambda` : L2 Regularization 적용 값으로 피처 개수↑ → 적용 검토, 값↑ → 과적합 감소 효과(default=1, alias: reg_lambda)

- `alpha` : L1 Regularization 적용 값으로 위와 동일 효과(default=0, alias: `reg_alpha`)
- `scale_pos_weight` : 치우친 비대칭 클래스로 구성된 데이터 세트의 균형 유지 위함(default=1)

3. 학습 테스트 파라미터: 학습 수행 시의 객체 함수 / 평가를 위한 지표 등을 설정 파라미터

- `objective` : 최솟값을 가질 손실 함수 정의
- `binary:logistic` : 이진 분류 시 적용
- `multi:softmax` : 다중 분류 시 적용하며 레이블 클래스 개수인 `num_class` 파라미터 지정 필요
- `multi:softprob` : 개별 레이블 클래스의 해당되는 예측 확률 반환
- `eval_metric` : 검증에 사용되는 함수 정의 (default=회귀 시 `rmse`, 분류 시 `error`)
 - `rmse` - root mean square error
 - `mae` - mean absolute error
 - `logloss` - negative log-likelihood
 - `merror` - binary classification error rate(0.5 threshold)
 - `mlogloss` - multiclass logloss
 - `auc` - area under the curve
- 과적합 문제 해결방안
 - eta 낮추기(0.01~0.1) & `num_round` / `n_estimators` 높이기
 - `max_depth` 낮추기
 - `min_child_weight` 높이기
 - `gamma` 값 높이기
 - `subsample` 과 `colsample_bytree` 조정하기

사이킷런 래퍼 XGBoost의 개요 및 적용

변경된 네이티브 하이퍼 파라미터:

- eta → learning_rate
- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha

+) n_estimators와 num_boost_round는 동일한 파라미터

7) LightGBM

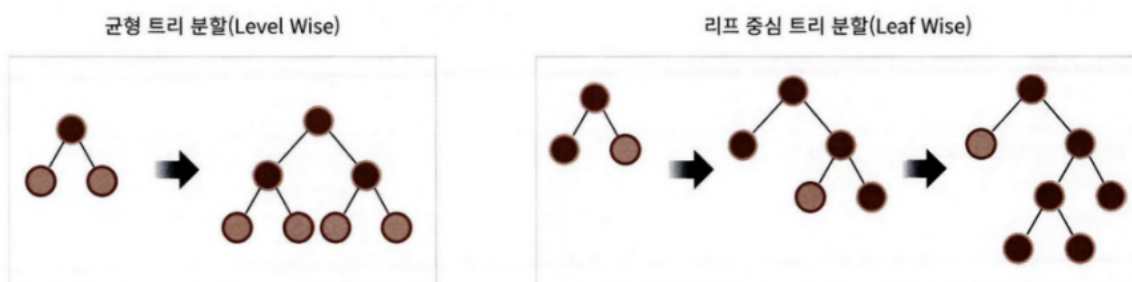
LightGBM 개요

[장점]

- XGBoost보다도 학습 시간↓
- 메모리 사용량 상대적으로 ↓
- 성능은 XGBoost와 별반 다르지 X
- XGBoost보다 기능상 다양성 ↑
- 카테고리형 피처의 자동 변환과 최적 분할

[단점]

- 적은 데이터 세트에서는 과적합 발생이 쉬움
- 리프 중심 트리 분할 방식 사용 → max delta loss를 가지는 노드를 지속적으로 분할
→ 트리 깊이↑ + 비대칭적 → 예측 오류 손실 최소화 가능(↔ 균형 트리 분할 방식 : 과적합에 강함)



패키지명: `lightgbm` -사이킷런 래퍼, 파이썬 래퍼 클래스 두 개 존재

LightGBM 하이퍼 파라미터

주요 파라미터:

- **num_iterations** [default = 100]: 반복 수행하려는 트리의 개수를 지정합니다. 크게 지정할수록 예측 성능이 높아질 수 있으나, 너무 크게 지정하면 오히려 과적합으로 성능이 저하될 수 있습니다. 사이킷런 GBM과 XGBoost의 사이킷런 호환 클래스의 **n_estimators**와 같은 파라미터이므로 LightGBM의 사이킷런 호환 클래스에서는 **n_estimators**로 이름이 변경됩니다.
- **learning_rate** [default = 0.1]: 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률을 값입니다. 일반적으로 **n_estimators**를 크게 하고 **learning_rate**를 작게 해서 예측 성능을 향상시킬 수 있으나, 마찬가지로 과적합 이슈와 학습 시간이 길어지는 부정적인 영향도 고려해야 합니다. GBM, XGBoost의 **learning rate**와 같은 파라미터입니다.
- **max_depth** [default=-1]: 트리 기반 알고리즘의 **max_depth**와 같습니다. 0보다 작은 값을 지정하면 깊이에 제한이 없습니다. 지금까지 소개한 Depth wise 방식의 트리와 다르게 LightGBM은 Leaf wise 기반이므로 깊이가 상대적으로 더 깊습니다.
- **min_data_in_leaf** [default = 20]: 결정 트리의 **min_samples_leaf**와 같은 파라미터입니다. 하지만 사이킷런 래퍼 LightGBM 클래스인 **LightGBMClassifier**에서는 **min_child_samples** 파라미터로 이름이 변경됩니다. 최종 결정 클래스인 리프 노드가 되기 위해서 최소한으로 필요한 레코드 수이며, 과적합을 제어하기 위한 파라미터입니다.
- **num_leaves** [default = 31]: 하나의 트리가 가질 수 있는 최대 리프 개수입니다.
- **boosting** [default = gbdt]: 부스팅의 트리를 생성하는 알고리즘을 기술합니다.
 - gbdt: 일반적인 그래디언트 부스팅 결정 트리
 - rf: 랜덤 포레스트
- **bagging_fraction** [default = 1.0]: 트리가 커져서 과적합되는 것을 제어하기 위해서 데이터를 샘플링하는 비율을 지정합니다. 사이킷런의 GBM과 **XGBClassifier**의 **subsample** 파라미터와 동일하기에 사이킷런 래퍼 LightGBM인 **LightGBMClassifier**에서는 **subsample**로 동일하게 파라미터 이름이 변경됩니다.
- **feature_fraction** [default = 1.0]: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율입니다. 과적합을 막기 위해 사용됩니다. GBM의 **max_features**와 유사하며, **XGBClassifier**의 **colsample_bytree**와 똑같므로 LightGBM **Classifier**에서는 동일하게 **colsample_bytree**로 변경됩니다.
- **lambda_l2** [default=0.0]: L2 regulation 제어를 위한 값입니다. 피처 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다. **XGBClassifier**의 **reg_lambda**와 동일하므로 LightGBMClassifier에서는 **reg_lambda**로 변경됩니다.
- **lambda_l1** [default = 0.0]: L1 regulation 제어를 위한 값입니다. L2와 마찬가지로 과적합 제어를 위한 것이며, **XGBClassifier**의 **reg_alpha**와 동일하므로 LightGBMClassifier에서는 **reg_alpha**로 변경됩니다.

Learning Task 파라미터:

- **objective**: 최솟값을 가져야 할 손실함수를 정의합니다. Xgboost의 objective 파라미터와 동일합니다. 애플리케이션 유형, 즉 회귀, 다중 클래스 분류, 이진 분류인지에 따라서 objective인 손실함수가 지정됩니다.

하이퍼 파라미터 튜닝 방안

- num_leaves의 개수를 중심으로 min_child(min_data_in_leaf), max_depth를 함께 조정 → 모델의 복잡도 줄이기
- learning_rate를 작게 하면서 n_estimators를 크게 하기 → n_estimators를 너무 크게 하면 과적합 발생 가능성을 유의
- reg_lambda, reg_alpha와 같은 regularization 적용
- conlsample_bytree, subsample 파라미터 적용 → 학습에 사용할 피쳐 개수나 데이터 샘플링 레코드 개수 줄이기

파이썬 래퍼 LightGBM과 사이킷런 XGBoost, LightGBM 하이퍼 파라미터 비교

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

8) 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

베이지안 최적화: 목적 함수 식을 제대로 알 수 없는 블랙 박스 형태의 함수에서 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식

- 특징: 베이지안 확률 기반
 - +) 베이지안 확률 : 새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선

- 구성 중요 요소

- 대체 모델 : 획득 함수로부터 최적 함수를 예측할 수 있는 입력 값을 추천 받음 → 이를 기반으로 최적 함수 모델 개선
- 획득 함수 : 개선된 대체 모델 기반으로 최적 입력값 계산

진행 과정

1. 최초 랜덤 하이퍼 파라미터 샘플링 & 결과 관측
2. 관측 값 기반으로 대체 모델이 최적 함수 추정
 - 예측 함수의 신뢰구간 포함(추정된 함수의 결괏값 오류 편차=함수의 불확실성)
 - 최적 관측값은 y축 value에서 가장 높은 값을 가질 때 하이퍼 파라미터
3. 추정된 최적 함수 기반으로 획득 함수가 다음 관측할 하이퍼 파라미터 값 계산, 이전의 최전 관측값보다 더 큰 최댓값을 가질 가능성 높은 지점을 찾아 , 대체 모델에 전달
4. 획득 함수로부터 전달된 하이퍼파라미터 수행해 관측된 값 기반으로 대체 모델 갱신, 다시 최적 함수 예측 추정

HyperOpt 사용하기

목적 함수 반환 값의 최솟값을 가지는 최적 입력값 유추

- HyperOpt 주요 로직

1. 입력 변수명과 입력값의 검색 공간 설정
 - 딕셔너리 형태로 설정: 키는 입력 변수명, value값은 검색 공간
 - hp 모듈 : 입력값의 검색 공간 다양 설정 가능하게 여러 함수 제공
- `hp.quniform(label, low, high, q)`: label로 지정된 입력값 변수 검색 공간을 최솟값 low에서 최댓값 high까지 q의 간격을 가지고 설정.
 - `hp.uniform(label, low, high)`: 최솟값 low에서 최댓값 high까지 정규 분포 형태의 검색 공간 설정
 - `hp.randint(label, upper)`: 0부터 최댓값 upper까지 random한 정숫값으로 검색 공간 설정.
 - `hp.loguniform(label, low, high)`: `exp(uniform(low, high))`값을 반환하며, 반환 값의 log 변환 된 값은 정규 분포 형태를 가지는 검색 공간 설정.
 - `hp.choice(label, options)`: 검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정. Options는 리스트나 튜플 형태로 제공되며 `hp.choice('tree_criterion', ['gini', 'entropy'])`과 같이 설정하면 입력 변수 `tree_criterion`의 값을 'gini'와 'entropy'로 설정하여 입력함.

2. 목적 함수 설정

인자 : 변숫값과 검색 공간을 가지는 딕셔너리로 특정 값을 반환하는 구조
반환값 : 단일값, 숫자형 외에도 딕셔너리 형태 가능

3. 목적 함수의 반환 최솟값을 가지는 최적 입력값 유추

- 베이زي안 최적화 기법에 기반하여 - fmin() 함수 제공
- fn: 위에서 생성한 objective_func와 같은 목적 함수입니다.
- space: 위에서 생성한 search_space와 같은 검색 공간 딕셔너리입니다.
- algo: 베이زي안 최적화 적용 알고리즘 입니다. 기본적으로 tpe.suggest이며 이는 HyperOpt의 기본 최적화 알고리즘인 TPE(Tree of Parzen Estimator)를 의미합니다.
- max_evals: 최적 입력값을 찾기 위한 입력값 시도 횟수입니다.
- trials: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용됩니다. Trials 클래스를 객체로 생성한 변수명을 입력합니다.
- rstate: fmin()을 수행할 때마다 동일한 결과값을 가질 수 있도록 설정하는 랜덤 시드(seed) 값입니다.

HyperOpt를 이용한 XGBoost 하이퍼 파라미터 최적화

1. 적용해야 할 하이퍼 파라미터와 검색 공간 설정
2. 목적 함수에서 XGBoost 학습 후 예측 성능 결과를 반환 값으로 설정
3. fmin() 함수에서 (목적 함수를 하이퍼 파라미터 검색 공간의 입력값들을 사용하여 최적의 예측 성능 결과를 반환하는) 최적 입력값들을 결정

+) 주의점:

정숫값 입력만 받는 하이퍼 파라미터: HyperOpt의 입력값과 반환값은 모두 실수형 → 하이퍼 파라미터 입력 시 형변환 필요

HyperOpt는 최솟값을 찾는 최적화 수행 → 최댓값을 찾고 싶으면 -1을 곱해야 함

10) 분류 실습 - 캐글 신용카드 사기 검출

- 불균형한 레이블
 - 0: 정상적인 신용카드 트랜잭션 데이터
 - 1: 사기 트랜잭션(0.172%)

언더 샘플링과 오버 샘플링의 이해

- 이상 레이블 D수 > 정상 레이블 D수

- 이상 레이블 D가 적어 제대로 다양한 유형 학습X(=정상 레이블로 치우친 학습 수행)
- 오버 샘플링
 - 적은 D 증식해 충분히 확보하는 방법
 - 동일X, 원본의 피쳐 값들을 약간만 변경해 증식
 - SMOTE : k-nn을 찾아 이 D와 k개 이웃들의 차이 → 일정 값 → 기존과 약간 차이나는 새 D생성
- 언더 샘플링
 - 많은 D를 적은 D 수준으로 감소

이상치 데이터 제거 후 모델 학습/예측/평가

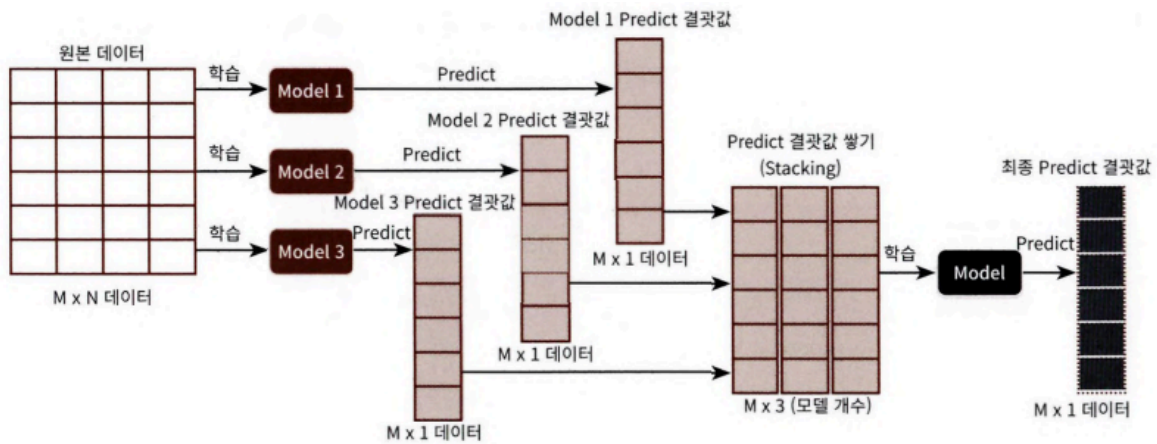
outlier : 전체 패턴에서 벗어난 이상 값 가진 D

IQR : 사분위 값의 편차를 이용해 outlier 제거

- Q1 ~ Q3 : IQR
 - $IQR * 1.5$ 로 생성된 범위 이용 → 최댓값/최솟값 결정 → 최댓값보다 큰 값 또는 최솟값보다 작은 값을 이상치 데이터로 간주

11) 스택킹 앙상블

- 예측값을 합하다 = 스택킹
- 개별 여러 알고리즘 결합 → 예측 결과 도출
- 배깅 / 부스팅과 차이점: 개별 알고리즘으로 예측한 데이터를 기반으로, 다시 예측 수행
 - 개별 알고리즘의 예측 결과 데이터 세트 → 최종 메타 데이터 세트 → 별도의 ML 알고리즘으로 최종 학습 수행 → 테스트 데이터 기반 다시 최종 예측 수행
 - 메타 모델 : 개별 모델의 예측된 데이터로 세트를 다시 기반으로 하여 학습하고 예측하는 방식
- 두 종류의 모델 필요: 개별적인 기반 모델 + 최종 메타 모델(개별 기반 모델의 예측 데이터를 학습 데이터로 만들어서 학습)
- 여러 개별 모델의 예측 데이터 → 각각 스택킹 형태로 결합 → 최종 메타 모델의 학습용 피쳐 데이터 세트 & 테스트용 피쳐 데이터 세트 생성



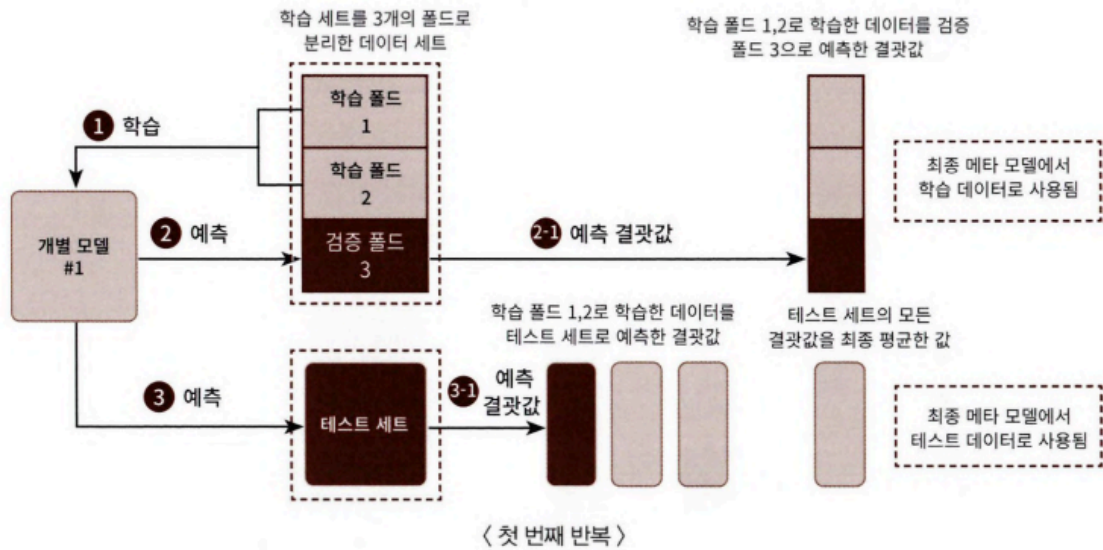
기본 스택킹 모델

1. 스택킹에 사용될 ml 알고리즘 클래스 생성
 - 개별 모델 : KNN, RF, DT, ADABOOST
 - 최종 모델 : Logistic Regression
2. 개별 모델 학습
3. 개별 모델의 예측 데이터 세트 반환, 개별 모델의 예측 정확도 확인
4. 개별로부터 얻은 예측값을 칼럼 레벨로 옆으로 붙여 피쳐 값으로 만들기
5. 최종 메타 모델에서 학습 데이터로 다시 사용

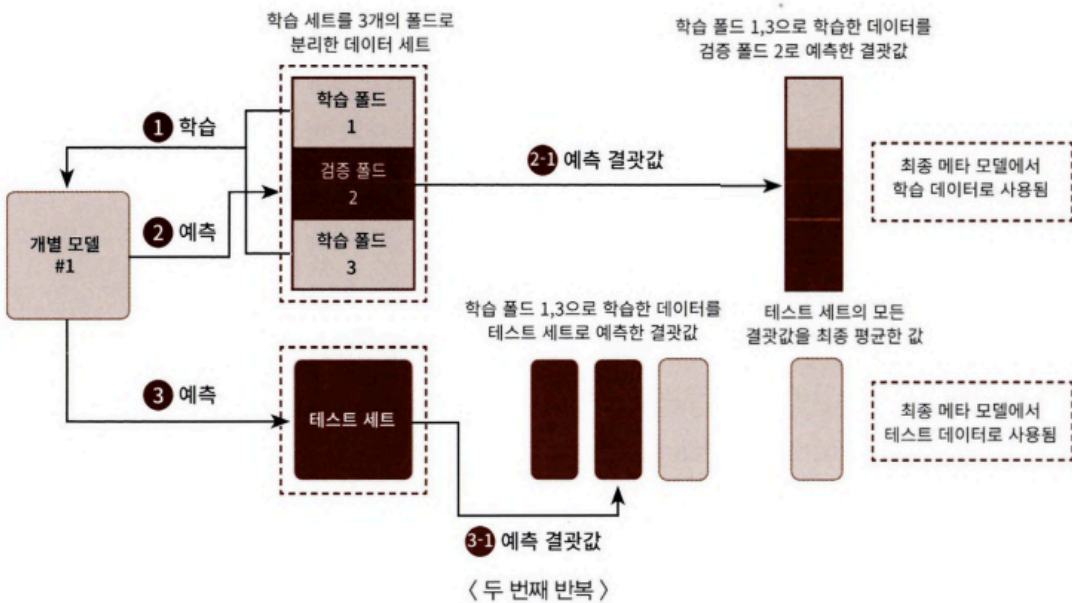
CV 세트 기반의 스택킹

- for 과적합 개선
- 최종 메타 모델을 위한 data set 만들 때, 교차 검증 기반으로 예측된 결과 데이터 세트 이용

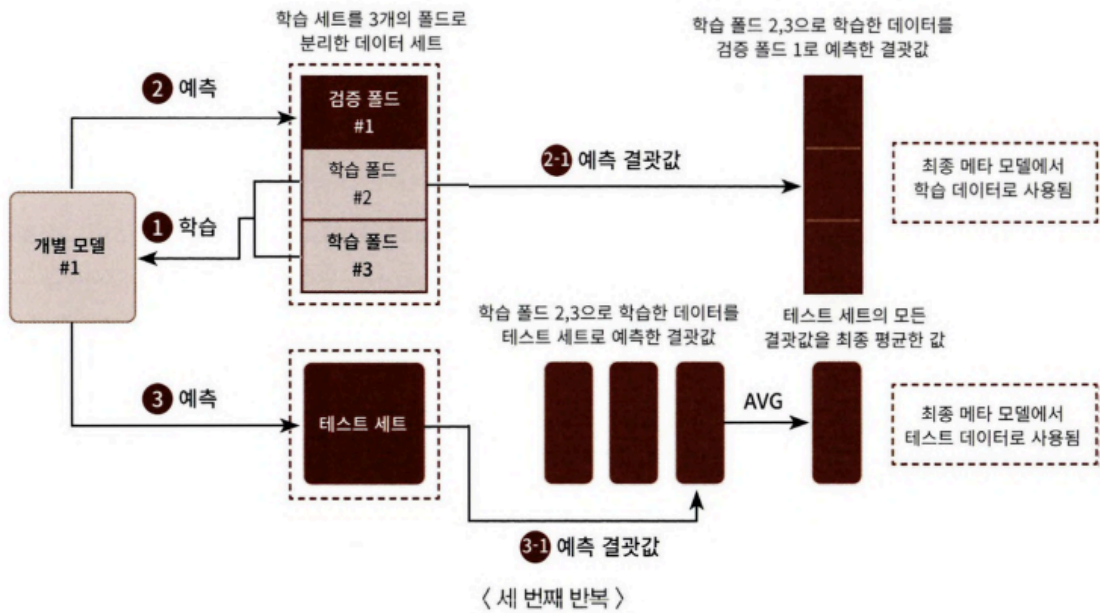
1. 각 모델별 예측 결과 값 기반 → 메타 모델 위한 train/test 데이터 생성



2. 1에서 개별 모델이 생성한 test/test 데이터를 모두 스택킹으로 합침 → 메타 모델이 최종 학습할 최종 train/test 데이터 생성



3. 최종 생성 train 데이터 + 원본 train 데이터의 레이블 기반 학습 → 최종 생성 test 데이터 예측 → 원본 test 데이터의 레이블 데이터 기반 평가



<전체 모델 도식화>

